

RepCore — Tenir sans payer

Analyse du stockage et de la bande passante · 23 septembre 2026 · relevé sur le dépôt au commit 74c38f6

1 · L'analyse

A — Le mur le plus proche n'est pas le stockage, c'est la bande passante d'hébergement

`app/index.html` pèse 6,15 Mo, soit **1,9 Mo une fois compressé**, et il est servi en `no-cache` — délibérément, pour que les mises en ligne arrivent tout de suite. Firebase Hosting en plan gratuit plafonne à **360 Mo transférés par jour**, soit **189 ouvertures d'application par jour, toutes personnes confondues**. Avec 467 mises en ligne sur les trente derniers jours, chaque build renvoie ce fichier entier à tous les appareils déjà installés. Le plafond n'est donc pas atteint par les athlètes qui paient : il est atteint par le rythme de développement, avant même d'avoir vendu.

B — Une vidéo supprimée dans l'app n'est jamais supprimée chez Cloudinary

`deleteVideo` (`app/index.html:90238`) retire l'entrée du dossier, pose une pierre tombale de synchronisation, et **laisse le fichier en ligne** — le code le dit lui-même ligne 90256. Pire : le compte Cloudinary par défaut est **codé en dur** (`dntu57mL` , ligne 104073), donc tous les coachs qui n'ont pas configuré le leur partagent **un seul plan gratuit de 25 crédits par mois**. Et comme la vidéo est le seul média qui part sans compression (100 à 300 Mo pour une série de 45 s), une centaine de vidéos suffit à saturer ce compte — **pour tout le monde en même temps**.

C — La base temps réel se réécrit en entier, et elle contient des images

Chaque enregistrement relit le dossier distant puis le **remet en entier** (`PUT` de `safe` , `app/index.html:17035`) : un enregistrement coûte deux fois le poids du dossier. Or les photos de bilan y entrent en base64 (`app/index.html:35657`), et les règles autorisent jusqu'à **420 000 caractères par image** (`database.rules.json:184`). Un dossier portant un an de bilans photo pèse plusieurs mégaoctets ; à 10 Go téléchargés par mois en plan gratuit, cela ne laisse **que quelques centaines d'enregistrements par mois, tous utilisateurs confondus**.

D — Rien n'expire, nulle part

Aucune rétention côté serveur : ni vidéo, ni photo, ni séance, ni bilan, ni le nœud `metrics` qui crée une clé par jour à vie. La seule purge qui existe (`purgerPhotosAnciennes` , `app/index.html:18307`) ne touche que le `localStorage` du téléphone, jamais le serveur. **Un coach à quinze athlètes pendant deux ans ne libère pas un octet**.

Ce qui est déjà bien fait, et qu'il ne faut pas défaire. Les photos de progression passent déjà par IndexedDB en Blob, jamais en base64 dans le dossier (`app/index.html:58751` et suivantes). La descente lit `users/<clé>/updatedAt.json` — une vingtaine d'octets — avant de descendre un dossier complet. Une sonde de quota mesure déjà les octets RTDB et dégrade la synchronisation de 5 à 30 minutes au-delà de 85 % (`app/index.html:14240`). Un plafond de 200 comptes coachs gratuits existe (`LIBRE_MAX` , ligne 14906). Le socle est sain : ce qui manque, c'est **l'expiration**.

2 · Les plafonds, en chiffres

RESSOURCE	PLAFOND GRATUIT	COÛT UNITAIRE ACTUEL	NOMBRE DE PERSONNES AVANT LE MUR
-----------	-----------------	----------------------	----------------------------------

Hosting bande passante	360 Mo / jour	1,9 Mo par ouverture après chaque mise en ligne	≈ 189 ouvertures/jour <i>≈ 60 utilisateurs si 3 builds/jour</i>
Hosting stockage	10 Go	46 Mo par déploiement	confortable
Realtime DB téléchargement	10 Go / mois	2 × le poids du dossier, à chaque enregistrement	≈ 10 coachs actifs <i>dossiers sans photos : ≈ 200</i>
Realtime DB stockage	1 Go	jusqu'à 420 Ko par photo de bilan	≈ 120 années-athlète
Cloudinary compte partagé	25 crédits / mois (≈ 25 Go cumulés)	100 à 300 Mo par vidéo, jamais libérés	≈ 100 vidéos, une seule fois <i>puis panne pour tous</i>
Firebase Storage PDF de programme	5 Go	1 fichier par athlète, écrasé	borné par construction

À vérifier avant toute décision de facture. Le dépôt contient huit Cloud Functions (`functions/index.js`) qui exigent le plan Blaze, mais `deploy.sh` et le workflow GitHub ne déploient que `hosting` et `database`. Ouvrir la console Firebase et relever le plan réel du projet `repcore-sync` : les chiffres ci-dessus supposent le plan gratuit.

3 · Ce que ça change pour le revenu

Un athlète paie 9,50 € par mois, un coach 19 € (quinze athlètes) ou 39 € (sans limite). Aujourd'hui, **le coût marginal d'un athlète est illimité dans le temps** : il dépose des vidéos qui ne partiront jamais et des photos qui grossissent un dossier réécrit en entier à chaque sauvegarde. Un athlète qui résilie au bout de trois mois continue de coûter, indéfiniment. Les quatre correctifs qui suivent ramènent ce coût à une valeur **bornée et prévisible** — ce qui est exactement la condition pour accepter du monde sans regarder la facture monter.

4 · Le prompt de mise en application

À coller tel quel dans une session Claude Code ouverte sur `C:\RepCore-web`.

MISSION – Borner le coût de stockage de RepCore, sans rien retirer à personne.

Contexte : RepCore tourne sur Firebase (Hosting + Realtime Database) et Cloudinary, en plans gratuits. Rien n'expire nulle part, et trois plafonds sont proches. Ta mission est de rendre le coût par athlète BORNÉ DANS LE TEMPS, sans jamais faire disparaître une donnée sans que la personne l'ait vue venir.

RÈGLE ABSOLUE, VALABLE POUR LES SIX LOTS

Aucune donnée ne disparaît en silence. Toute expiration doit être :

1. ANNONCÉE à l'écran avant de survenir (un compte à rebours visible) ;
2. RÉCUPÉRABLE par un geste de la personne (épingleur, télécharger) ;
3. JOURNALISÉE localement, pour qu'un « où est passée ma vidéo ? » ait une réponse.

Une purge muette est un défaut, pas une optimisation. Si tu dois choisir entre économiser un octet et prévenir quelqu'un, préviens.

Respecte les contraintes du dépôt : `app/index.html` est en CRLF (éditer en Python en comptant les LF, jamais avec `sed -i` ni patch), la suite de tests doit rester au vert, et tout nouveau domaine appelé doit être déclaré dans `privacy.html`.

== LOT 1 – L'APP NE DOIT PLUS SE RETÉLÉCHARGER EN ENTIER (le plus urgent) ==

Problème : `app/index.html` fait 1,9 Mo compressé, est servi en no-cache, et 467 mises en ligne ont eu lieu en trente jours. À 360 Mo/jour de plafond Hosting, cela fait 189 ouvertures d'app par jour pour tout le monde.

À faire :

- Sortir de `index.html` les blocs les plus lourds et les plus stables vers des fichiers versionnés par leur nom (ex. `rc-core.<RC_BUILD>.js`). Vise les plus gros blocs figés en premier : tables d'exercices, textes longs, styles. Ces fichiers-là sont servis en `Cache-Control: public, max-age=31536000, immutable` : ils ne repartent JAMAIS deux fois sur le réseau. Seul le squelette `index.html` reste en no-cache.
- Mesure obligatoire AVANT/APRÈS : donne-moi le poids gzippé de `index.html` dans les deux cas, et le nombre d'ouvertures/jour que cela autorise. Objectif : `index.html` sous 300 Ko compressés.
- Ajouter à `app/sw.js` une purge des anciennes versions de ces fichiers à l'activer : garder la version courante et la précédente, supprimer le reste du cache.
- NE PAS toucher aux en-têtes `/app`, `/i` et `/app/sw.js` dans `firebase.json` : les commentaires y expliquent quatre lots de dégâts déjà payés.

== LOT 2 – SUPPRIMER POUR DE VRAI CHEZ CLOUDINARY ==

Problème : `deleteVideo` (`app/index.html:90238`) laisse le fichier en ligne. Le commentaire ligne 90256 l'assume. Rien n'est jamais libéré.

À faire :

- Ajouter dans `functions/index.js` une fonction appellable `cloudinaryDestroy`, sur le modèle exact de `getCloudinarySignature` (secrets `CLOUDINARY_API_SECRET` / `_API_KEY`, refus si `request.auth` absent). Elle vérifie que le `publicId` commence par `'repcore/<id de l'appelant>/'` OU que l'appelant est le coach désigné par le dossier, puis appelle `/destroy` avec `resource_type` correct (video ou image).
- Brancher `deleteVideo` dessus. Si l'appel échoue, la vidéo est tout de même retirée de l'app, mais le `publicId` est enfilé dans une file `rc_cloudinary_a_purger` rejouée au démarrage suivant – un fichier orphelin ne doit jamais rester orphelin en silence.
- Même traitement pour les photos de progression déjà partagées (`phpRevoquer` existe déjà, ligne 59009 : elle retombe sur le `delete_token` de dix minutes, donc elle échoue au-delà – la faire retomber sur `cloudinaryDestroy`).
- Prévoir la migration de l'existant : un script `scripts/purge_cloudinary_orphelins.py`

qui liste les `public_id` du compte, les recoupe avec les dossiers RTDB, et écrit un RAPPORT. Il ne supprime rien tout seul : il produit une liste que je valide.

== LOT 3 – UNE RÉTENTION, ANNONCÉE ET RÉCUPÉRABLE ==

Problème : aucune donnée n'expire. Un athlète résilié coûte indéfiniment.

À faire :

- Vidéos de séance : conservation **90 jours** par défaut (pas 30 : un cycle d'entraînement fait souvent 12 semaines, et une correction technique se compare d'un bout du cycle à l'autre). Au-delà, la vidéo est détruite chez Cloudinary et l'entrée garde `{nom, date, expiree:true}` – le coach voit qu'elle a existé.
- ÉPINGLEAGE : le coach comme l'athlète peuvent marquer une vidéo « à garder ». Une vidéo épinglée n'expire jamais. Plafond : 10 épingles par athlète, dit à l'écran.
- AVERTISSEMENT : à J-7, la vidéo porte un bandeau « expire dans 7 jours · Garder · Télécharger ». Le téléchargement récupère le fichier avant destruction.
- Athlète résilié ou sans accès depuis 180 jours : ses médias Cloudinary sont détruits après un avertissement par écran à l'ouverture, 30 jours avant. Le dossier texte, lui, RESTE – il ne pèse presque rien et c'est son historique d'entraînement.
- Nœud `metrics` : une clé par jour à vie. Ajouter une purge glissante à 400 jours.

== LOT 4 – SORTIR LES IMAGES DE LA BASE TEMPS RÉEL ==

Problème : les photos de bilan partent en base64 dans `users/<clé>` (ligne 35657) et les règles autorisent 420 000 caractères par image (`database.rules.json:184`). Or chaque enregistrement RELIT puis RÉÉCRIT le dossier entier (PUT de safe, ligne ~17035) : un dossier de 5 Mo coûte 10 Mo par sauvegarde.

À faire :

- Reprendre EXACTEMENT le patron déjà écrit pour les photos de progression (`compressImageBlob` → `IndexedDB` en Blob → upload Cloudinary à la demande, lignes 58745 à 58935). Ce module est bien fait : il ne met dans le dossier que `{cle, w, h, octets}` et, après partage explicite, `{url, publicId}`.
- Migrer les photos de bilan dessus. Les base64 déjà en base sont convertis à la volée, une fois, au premier chargement du dossier : lire, téléverser, remplacer par l'URL, pousser. Avec un journal, et sans jamais perdre l'image si l'upload échoue.
- Une fois la migration faite et vérifiée, abaisser les `.validate` de 420000 à 600 (la longueur d'une URL) dans `database.rules.json`. Pas avant : une règle plus stricte que les données déjà en place ferait échouer tous les enregistrements.
- Mesure : donne-moi le poids médian d'un dossier avant/après.

== LOT 5 – COMPRESSER LA VIDÉO AVANT L'ENVOI ==

AUDIT_ENVOI_VIDEO.md décrit déjà la solution en détail et elle n'a pas été appliquée : transcodage 720p / 2,5 Mbit/s en WebCodecs, repli MediaRecorder, jamais ffmpeg.wasm. C'est un facteur 6 à 20 sur le poids ET sur la durée d'envoi. Applique-la telle qu'elle est écrite dans ce document. Elle change à elle seule l'ordre de grandeur de tout le lot 3 : à 18 Mo la minute au lieu de 170, la rétention 90 jours devient confortable au lieu d'être tendue.

== LOT 6 – VOIR VENIR LE MUR, POUR DE VRAI ==

Problème : `etatQuota()` (ligne 14240) ne mesure que l'appareil courant. Elle ne voit jamais le total réel, et elle ne couvre que RTDB – ni Hosting, ni Cloudinary.

À faire :

- Agréger les compteurs par jour dans le nœud `metrics` (incréments serveur `.sv`, comme `rcm()` le fait déjà) : octets RTDB entrants/sortants, nombre d'uploads Cloudinary, octets Cloudinary envoyés. Aucune donnée personnelle, juste des nombres.
- Un écran d'administration, visible de moi seul, qui affiche pour chaque ressource : consommé du mois, plafond, pourcentage, et la DATE projetée de dépassement à pente constante (`projectionQuota` existe déjà ligne 14320, réutilise-la).
- Étendre la dégradation existante : au-delà de 85 %, en plus de la synchro qui passe à 30 minutes, l'envoi de vidéo affiche « le service est en charge, ton envoi peut attendre ce soir » et propose de différer – sans jamais l'interdire.

== ORDRE ET LIVRAISON ==

Fais les lots dans cet ordre : **1, 2, 5, 3, 4, 6**. Le lot 1 écarte le mur le plus proche ; le 2 arrête l'hémorragie ; le 5 divise le problème par dix avant qu'on écrive la moindre règle de rétention.

Un lot = un commit = un build vérifié en ligne. Pour chaque lot :

- la suite de tests au vert, sans exception passée sous silence ;
- une mesure AVANT/APRÈS en octets, pas une promesse ;
- commit + push sur main, puis vérification que le RC_BUILD servi en ligne est bien le nouveau – pas que le dépôt l'a reçu.

Ne me donne pas de liste de courses : fais-le.